TCI Protocol Overview

TCI Protocol — Overview & Implementation Comparison

TCI (Transceiver Control Interface) is a WebSocket-based protocol created by Expert Electronics for their ExpertSDR2/3 software. It has been adopted by several open-source SDR applications, each with their own extensions or deviations from the reference implementation.

> **Goal of this repository:** Document the TCI protocol and highlight the differences between known implementations, so client developers know what to expect when connecting to various servers.

Table of Contents

1. Protocol Overview
2. Connection & Handshake
3. Command Format
4. Command Reference
5. Binary Frame Format
6. Implementation Comparison
7. Known Extensions
8. Resources
9. Conclusion: Is a Universal TCI Implementation Possible?

Protocol Overview

Property	Value
Transport	WebSocket (RFC 6455)
Default port	`50001`
Text frames	Commands (UTF-8)
Binary frames	IQ data, audio data, spectrum data
Protocol version	`2` (current)
Direction	Bidirectional; server echoes state to all clients

TCI is designed as a **remote control interface** for SDR transceivers. A server (SDR application) exposes a WebSocket endpoint; one or more clients connect and control the radio. State changes are broadcast to **all** connected clients.

Connection & Handshake

After establishing a WebSocket connection, the server immediately sends an **init block** — a sequence of commands describing the radio's capabilities. The block always ends with `ready` (no semicolon).

```
protocol:2;
device:ExpertSDR2;
receive_only:false;
trx_count:1;
```

```
channels_count:2;
vfo_limits:100000,450000000;
if_limits:-96000,96000;
modulations_list:AM,SAM,DSB,LSB,USB,CWL,CWU,NFM,DIGL,DIGU,SPEC,DRM;
iq_samplerate:192000;
audio_samplerate:48000;
tx_profiles_ex:Default,Contest,DX;
ready
```

Init field	Description
`protocol`	TCI protocol version (currently `2`)
`device`	Software/device name string
`receive_only`	`true` if TX is not available
`trx_count`	Number of independent transceivers
`channels_count`	Number of RX channels per TRX
`vfo_limits`	Min/max VFO frequency in Hz
`if_limits`	IF passband limits in Hz
`modulations_list`	Comma-separated list of supported modes
`iq_samplerate`	IQ stream sample rate (Hz)
`audio_samplerate`	Audio stream sample rate (Hz)
`tx_profiles_ex`	Named TX profiles
`ready`	End of init block (no `:` or `;`)

After `ready`, the server will accept commands. The client may also send `start` to begin streaming and `stop` to end streaming.

Bare messages (no colon/semicolon)

```
ready    ← server only: end of init block
start    ← client: begin streaming
stop     ← client: stop streaming
```

Command Format

All TCI commands follow this syntax:

```
command_name:param0,param1,...,paramN;
```

- All lowercase
- Parameters separated by commas
- Terminated by semicolon
- No spaces

Query form — send with TRX index only (no value) to request current state:

```
vfo:0,0;        → server replies with vfo:0,0,14074000;
modulation:0;   → server replies with modulation:0,USB;
```

Server response — the server echoes the full command with current value(s) to all clients.

Command Reference

Frequency / VFO

Command	Parameters	Description
`vfo`	`trx, channel, freq_hz`	Set VFO frequency
`dds`	`trx, freq_hz`	Set DDS (center) frequency
`if`	`trx, channel, freq_hz`	Set IF frequency
`vfo_lock`	`trx, channel, true|false`	Lock VFO
`lock`	`trx, true|false`	Lock all VFOs on TRX
`split_enable`	`trx, true|false`	Enable split operation
`vfo_swap_ex`	`trx`	Swap VFO A/B
`rx_enable`	`trx, channel, true|false`	Enable/disable RX channel

Modulation & Filters

Command	Parameters	Description
`modulation`	`trx, mode`	Set modulation (USB/LSB/AM/SAM/DSB/CWL/CWU/NFM/DIGL/DIGU/SPEC/DRM)
`rx_filter_band`	`trx, low_hz, high_hz`	Set RX passband filter
`tx_filter_band_ex`	`trx, low_hz, high_hz`	Set TX passband filter

RIT / XIT

Command	Parameters	Description
`rit_enable`	`trx, true|false`	Enable RIT
`rit_offset`	`trx, offset_hz`	Set RIT offset
`xit_enable`	`trx, true|false`	Enable XIT
`xit_offset`	`trx, offset_hz`	Set XIT offset

DSP — Noise Reduction & Filtering

Command	Parameters	Description
`rx_nb_enable`	`trx, channel, true|false`	Noise Blanker 1
`rx_nb2_enable`	`trx, channel, true|false`	Noise Blanker 2
`rx_nb_enable_ex`	`trx, channel, nb_idx, true|false`	Extended NB (indexed)
`rx_bin_enable`	`trx, channel, true|false`	Binaural audio
`rx_anf_enable`	`trx, channel, true|false`	Auto Notch Filter
`rx_nr_enable`	`trx, channel, true|false`	Noise Reduction
`rx_nr_enable_ex`		Extended NR (indexed, 0–4)

Command	Parameters	Description
	`trx, channel, nr_idx, true|false`	

AGC / Squelch

Command	Parameters	Description
`agc_auto_ex`	`trx, channel, true|false`	Auto AGC
`agc_gain`	`trx, channel, gain_db`	Manual AGC gain
`sql_enable`	`trx, channel, true|false`	Squelch enable
`sql_level`	`trx, channel, level_db`	Squelch threshold

PTT / TX

Command	Parameters	Description
`trx`	`trx, true|false`	TX/RX toggle (PTT)
`tune`	`trx, true|false`	Tune (carrier)
`drive`	`trx, level_pct`	TX drive level (0–100)
`tune_drive`	`trx, level_pct`	Tune drive level
`tx_antenna`	`trx, ant_idx`	Select TX antenna
`rx_antenna`	`trx, channel, ant_idx`	Select RX antenna

Audio

Command	Parameters	Description
`rx_volume`	`trx, channel, volume_db`	RX audio volume
`mute`	`trx, true|false`	Mute audio
`mon_enable`	`trx, true|false`	TX monitor enable
`mon_volume`	`trx, volume_db`	TX monitor volume
`line_in`	`trx, level_db`	Line-in level

IQ & Audio Streaming

Command	Parameters	Description
`iq_start`	`trx`	Begin IQ stream for TRX
`iq_stop`	`trx`	Stop IQ stream
`audio_start`	`trx, channel`	Begin audio stream
`audio_stop`	`trx, channel`	Stop audio stream
`audio_stream_sample_type`	`trx, float32|int16`	Audio sample format
`audio_stream_channels`	`trx, 1|2`	Mono or stereo
`audio_stream_samples`	`trx, count`	Samples per frame
`audio_samplerate`	`trx, rate_hz`	Audio sample rate

Command	Parameters	Description
`tx_stream_audio_buffering`	`trx, true|false`	TX audio buffering

Sensors (server → client)

Command	Parameters	Description
`rx_smeter`	`trx, channel, level_dbm`	S-meter reading (dBm)
`tx_sensors`	`trx, mic_db, fwd_pwr, rev_pwr, swr`	TX sensor readout
`tx_swr`	`trx, swr`	SWR reading
`tx_forward_power`	`trx, watts`	TX forward power
`rx_channel_sensors`	`trx, channel, ...`	Extended RX sensors
`rx_sensors_enable`	`trx, true|false`	Enable RX sensor stream
`tx_sensors_enable`	`trx, true|false`	Enable TX sensor stream

Aliases (accepted by some implementations): - `s_meter` / `smeter` → `rx_smeter` - `tx_power` → `tx_forward_power` - `swr` → `tx_swr`

TX Profiles

Command	Parameters	Description
`tx_profiles_ex`	*(init only)*	List of profile names
`tx_profile_ex`	`trx, profile_name`	Select TX profile

CAT Pass-through

Command	Parameters	Description
`run_cat_ex`	`trx, cat_string`	Send CAT command to radio

Keepalive

Command	Parameters	Description
`keepalive`	*(no reply)*	Keep connection alive

Binary Frame Format

Binary WebSocket frames carry IQ data, audio data, and spectrum data. Each frame starts with a **64-byte header**.

Header Layout

```
Offset  Size  Field
------  ----  -----
0       1     flags / stream type
1       3     sample_rate (uint24 LE) — for IQ frames
4       4     header field 1
8       4     header field 2
12      4     sample_count (uint32 LE)
```

```
16      1     frame_type
17      40    reserved / padding
57      7     (end of 64-byte header)
64+         payload
```

> **Note:** Exact header field positions vary between implementations. The layout above is based on the ON7OFF reference and may differ for Thetis or other servers. Always validate against the specific server you are connecting to.

Frame Types

frame_type	Content	Format
`0`	IQ data	float32 interleaved I/Q pairs
`1`	RX audio	float32 stereo PCM
`2`	TX audio	float32 stereo PCM
`3`	Chrono / timing	—

IQ Data

- Interleaved float32: `[I0, Q0, I1, Q1, ...]`
- Sample rate: ≥ 96 kHz (192 kHz recommended for wide spectrum)
- One stream per TRX (started with `iq_start:<trx>;`)

Audio Data

- float32 stereo PCM at 48 kHz
- Started with `audio_start:<trx>,<channel>;`
- TX audio sent from client to server in same binary format

Implementation Comparison

"✓" = confirmed, "×" = not supported, "?" = unknown/not verified.

Part A — ExpertSDR · Thetis · deskHPSDR · Lyra-SDR · SDROxide

Feature	ExpertSDR	Thetis (ramdor)	Thetis (mi0bot)	deskHPSDR	Lyra-SDR-cpp	SDROxide
Role	Server	Server	Server	Server	Server	Client + Server
Hardware	SunSDR2 / Expert	ANAN (OpenHPSDR)	Hermes Lite 2	ANAN / HL2 (P1/P2)	HL2, BrickSDR2	Many (SoapySDR)
Language	Closed source	C# / .NET 4.8	C# / .NET	C	C++23 / Qt6	Rust
OS	Windows	Windows	Windows	macOS / Linux / Win	Win (Linux planned)	Linux / macOS / Win
Protocol version	2	2 (emulated)	2 (emulated)	2	1.9 / 2	2 (as client)
`device:` value	`ExpertSDR2/3`	`ExpertSDR2`	`ExpertSDR2`	?	?	?
`receive_only`	✓	✓	✓	?	?	?
	✓	✓	?	✓	?	?

Feature	ExpertSDR	Thetis (ramdor)	Thetis (mi0bot)	deskHPSDR	Lyra-SDR-cpp	SDROxide
`trx_count > 1`						
`channels_count`	✓	✓	?	✓	?	?
`tx_profiles_ex`	✓	✓	?	?	?	?
IQ streaming	✓ 192 kHz	✓	✓	✓ 48–384 kHz	✓	✓
RX audio streaming	✓ 48 kHz	✓	✓	✓	✓ (optional)	✓
TX audio streaming	✓	✓	✓	✓	?	?
Spectrum / FFT	✓ (proprietary)	?	?	?	?	?
`split_enable`	Standard	Numeric = query†	?	?	?	?
`rx_antenna`	✓	Via CAT (ZZOA)	?	?	?	?
`tx_antenna`	✓	Via CAT (ZZOC)	?	?	?	?
`run_cat_ex`	✓	✓ (ZZ dialect)	✓	?	?	Partial
ZZ CAT commands	×	✓ (ZZSO/ZZOA/ZZOC)	✓	×	×	×
`rx_nb_enable_ex`	✓	?	?	✓ (NB1/NB2)	?	?
`rx_nr_enable_ex`	NR 0–4	?	?	NR1–NR4+NNR	?	5 NR engines
`keepalive`	✓	✓	✓	?	?	?
`start` / `stop`	✓	✓ (radio power)	✓	?	?	?
Multi-client	✓	✓	✓	?	?	✓
WSS (TLS)	?	Planned	?	?	?	?
State echo to all	✓	✓	✓	?	?	?
Default TCI port	50001	50001	50001	?	?	?
TCI dispatch cmds	~all	many	many	?	?	?

Part B — NereusSDR · Kainos · Bolt SDR · Rigflow · OpenHPSDR Zeus

Feature	NereusSDR	Kainos	Bolt SDR	Rigflow	Zeus (archived)
Role	Server	Server	Server	Server (TCI client for digi)	Server
Hardware	ANAN, HL2 (P1/P2)	Hermes Lite 2	Hermes Lite 2	HL2, RTL-SDR	HL2, ANAN G2/G2 MkII
Language	C++20 / Qt6	C# / .NET	C# / .NET + Python	Rust	C# / .NET 10
OS	macOS / Linux / Win	Windows	Linux / macOS / Win	Linux / macOS	Linux / macOS / Win
Protocol version	2 (Thetis-faithful)	2 (Thetis-faithful)	?	? (client only)	ExpertSDR3 2.0
`device:` value	?	`Thetis` (intentional)	?	N/A	`Zeus`

Feature	NereusSDR	Kainos	Bolt SDR	Rigflow	Zeus (archived)
`receive_only`	✓	✓	?	N/A	✓
`trx_count > 1`	✓ (slices A/B/C/D)	✓	?	N/A	✓
`channels_count`	✓	✓	?	N/A	✓
`tx_profiles_ex`	?	?	?	N/A	?
IQ streaming	✓ (IQSwap + flag)	✓	?	N/A	✓ per-session
RX audio streaming	✓ (8–48 kHz WDSP)	✓	?	N/A	✓ 8–48 kHz
TX audio streaming	✓ (ring buffer)	✓	?	N/A	✓ via `tci` source
Spectrum / FFT	✓ (3-priority queue)	?	?	N/A	?
`split_enable`	Standard	Standard†	?	N/A	✓
`rx_antenna`	?	Via CAT (ZZOA)	?	N/A	✓
`tx_antenna`	?	Via CAT (ZZOC)	?	N/A	✓
`run_cat_ex`	?	✓ (ZZ dialect)	?	N/A	?
ZZ CAT commands	×	✓ (Thetis compat)	×	N/A	×
`rx_nb_enable_ex`	✓ (WDSP)	?	?	N/A	✓ (`rx_nb_enable`)
`rx_nr_enable_ex`	✓ (WDSP)	?	?	N/A	✓ (`rx_nr_enable`)
`keepalive`	✓	✓	?	N/A	?
`start` / `stop`	✓	✓	?	N/A	?
Multi-client	✓ (broadcast all)	✓	?	N/A	✓
WSS (TLS)	?	?	?	N/A	?
State echo to all	✓	✓	?	N/A	✓
Default TCI port	?	50001	40001	40001	50001
TCI dispatch cmds	62 (8 families)	many (Thetis-compat)	`trx` confirmed	?	~50+

†) In some Thetis/Kainos versions, `split_enable:0;` (numeric 0 instead of boolean) is used to query the current state.

Notable Deviations per Implementation

Thetis (ramdor & mi0bot)

- Emulates `device:ExpertSDR2;` even though hardware is OpenHPSDR
- Antenna selection uses **CAT pass-through** (`run_cat_ex`) with Kenwood-style ZZ commands:
  - `ZZOA` — select RX antenna
  - `ZZOC` — select TX antenna
  - `ZZSO` — squelch source
- `split_enable` query uses numeric `0` instead of omitting value
- Some parameters differ from ExpertSDR reference in subtle ways (filter bandwidth limits, NB threshold ranges)
- Depends on **WDSP** (Warren Pratt's DSP library) for all DSP processing

deskHPSDR (dl1bz)

- C implementation for macOS, Linux, Windows (cross-platform focus)
- Supports OpenHPSDR P1 and P2 protocols
- Extended IQ sample rates: 48 / 96 / 192 / 384 kHz
- DSP labels differ: “NR1”, “NR2”, “NR3”, “NR4”, “NNR” instead of indexed `rx_nr_enable_ex`
- Also uses WDSP

Lyra-SDR-cpp (N8SDR1)

- Modern C++23 codebase with Qt6 UI
- Targets Hermes Lite 2 and BrickSDR2 hardware
- Protocol versions 1.9 / 2.0 supported
- Currently Windows only; Linux/macOS planned

SDROxide (dividebysandwich)

- Written in Rust; cross-platform
- Acts as **both TCI client** (connects to ExpertSDR3) **and TCI server** (exposes its own endpoint)
- Multi-backend: SoapySDR, HPSDR, and others
- Partial CAT pass-through support when acting as client

NereusSDR (boydsoftprez)

- C++20/Qt6 port of Thetis; GPL-3.0; macOS, Linux, Windows
- TCI v2.0 server shipped in **v0.5.0** (2026-05-13) as Phase 3J-1
- **62 dispatch commands** across 8 families, with an init burst of ~98 wire frames matching Thetis
- Documented deviations from Thetis:
    1. Init-burst typo fix (one corrected field)
    2. Uses Qt6 `QWebSocketServer` instead of hand-rolled RFC 6455 framing
    3. Slices shown as A/B/C/D in UI but appear as `trx:N` on the wire
    4. UTF-8 enforced outbound
    5. Single-class `ClientChainApplet`
    6. Sensor-interval aggregation
    7. Broadcast to all connected clients
    8. Bind-interface dropdown (configurable network interface)
    9. DIGU/DIGL F5 default restoration
- RX audio resampled via WDSP at 8, 12, 16, 44.1, 48 kHz (multi-rate support)
- TX audio gated by single-client mutex with ring buffer
- Three-priority send queue: status → sensor → spectrum
- Layer-3 VFO coalescer for wheel-acceleration handling
- ~80-row verification matrix (`tests/data/tci/matrix.csv`)
- `nereusd` headless Core for remote operation; iPhone/iPad client available

Kainos (PNWHam-K7JUS)

- Windows-only fork of Thetis (and Thetis HL2 edition), targets Hermes Lite 2
- Current version 1.0.3 based on Thetis 2.10.3.15; builds with Visual Studio 2026
- Intentionally identifies as `Thetis` to TCI and TCP CAT clients for compatibility
- Inherits all Thetis TCI behaviour including ZZ CAT pass-through
- Adds dark UI, SmartSDR-style slice flags, built-in RTTY/CW tools, DX cluster, POTA spots, KiwiSDR list
- FreeDV RADE support via CMake-built RADE libraries
- Settings stored separately from Thetis; one-time import from existing Thetis install

Bolt SDR (pe5jw)

- Browser-based SDR frontend for Hermes Lite 2, built on top of station-engine (Zeus fork)
- C#/.NET backend + React web frontend; Python `tci-bridge` helper app
- TCI server on **port 40001** (non-standard); also exposes CAT TCP on port 19090
- Confirmed TCI command: `trx` (PTT via tci-bridge)
- Full command set not documented; protocol version not stated
- Also supports OpenHPSDR Protocol 1 and 2

Rigflow (dbourgoyne)

- Written in Rust; server runs on Linux/Raspberry Pi, client on Linux/macOS
- Server owns hardware and DSP; communicates with client via WebSocket + UDP media
- **TCI is used as a client interface only** — for digital modes (FT8/WSJT-X):
  - Linux: virtual audio (PipeWire/Pulse) or TCI
  - macOS: TCI only (labelled experimental)
- TCI server runs at `ws://127.0.0.1:40001`; started automatically when client is running
- Supported hardware: Hermes Lite 2 (primary), RTL-SDR (RX-only), Hardrock-50 amplifier

OpenHPSDR Zeus (brianbruff) — Archived

- Browser-based SDR console (.NET 10 backend, React/WebGL frontend)
- **Full TCI server implemented** (`Zeus.Server.Hosting/Tci/`, ~2100 lines C#) — earlier analysis was incorrect
- TCI protocol version `ExpertSDR3 2.0`; device name `"Zeus"`
- Handles: `trx`, `vfo`, `modulation`, `agc_mode`, `iq_start/stop`, `audio_start/stop`, `rx_filter_band`, `split_enable`, `rx_nr_enable`, `rx_nb_enable`, `rx_anf_enable`, `cw_macros`, TX audio injection via `tci` source arg, spot commands, and more
- Modulation: sends `CWL`/`CWU` correctly (NOT collapsed to `CW`); sends `FM` (accepts `NFM` alias); case-insensitive input, uppercase output
- AGC: uses `agc_mode` with `FIXED/LONG/SLOW/MEDIUM/FAST/CUSTOM`; accepts `OFF` → `FIXED`, `NORMAL`/`MED` → `MEDIUM`
- `iq_start`/`audio_start`: per-session subscription model (not exclusive ownership); supports 2-arg `iq_start:rx,bool;` form
- `trx`: supports 3-arg `trx:rx,bool,signal_source;` where `signal_source ∈ {tci, mic1, mic2, micPC, ecoder2}`
- Archived on 2026-07-17; code preserved; forks (AnanCore, station-engine, Bolt SDR) continue development
- Hardware: Hermes Lite 2 (P1), ANAN G2/G2 MkII (P2), ANAN-100D/Angelia (P2)

AnanCore (ChasingCoffee)

- Browser-based SDR console forked from OpenHPSDR Zeus; targets ANAN G2 Ultra on Raspberry Pi 5
- Inherits Zeus TCI server; TCI support **likely present** (Zeus fork); not independently verified
- Self-hosted single-file relay for remote operation
- Hardware: ANAN G2 Ultra (primary), Hermes Lite 2 and older ANAN boards

station-engine (Zeus-SDR)

- Headless .NET 10 OpenHPSDR station process (C#); forked from Zeus; radio transport, DSP, TX control
- Inherits Zeus TCI server; TCI support **likely present** (Zeus fork); not independently verified
- Designed to be used with a separate proprietary Zeus SDR client
- Runs on macOS, Windows, Linux (including ARM64 / Raspberry Pi)
- Hardware: OpenHPSDR P1 and P2

Known Extensions

ON7OFF Compactor Extensions

The ON7OFF TCI Remote client documents several **non-standard extensions** used by its own "Compactor" server component. These are **not** part of the official TCI spec and are not implemented by other servers.

Extension	Description
`compactor:1,...`	Compactor server identification
`qping` / `qpong`	Latency measurement
`rn_*` messages	Compactor-specific remote control
`TCRS` binary magic	Compressed spectrum / FFT data
`TCRC` binary magic	Compressed IQ data
`TCRA` binary magic	Opus-encoded audio
`TCRM` binary magic	TX monitor audio

Thetis CAT Extensions

Thetis maps radio functions not available in standard TCI to Kenwood ZZ CAT commands via `run_cat_ex`. Client software targeting Thetis must implement CAT parsing for these responses.

Resources

Resource	URL
Official TCI spec (Expert Electronics)	https://github.com/ExpertSDR3/TCI
Expert Electronics GitHub org	https://github.com/ExpertSDR3
ON7OFF TCI Protocol Reference	https://pure-editions.com/on7off/TCI-Remote/TCI_Protocol_Reference.html
Thetis (ramdor fork)	https://github.com/ramdor/Thetis
Thetis (mi0bot / HL2 fork)	https://github.com/mi0bot/OpenHPSDR-Thetis
deskHPSDR (dl1bz)	https://github.com/dl1bz/deskhpsdr
Lyra-SDR-cpp (N8SDR1)	https://github.com/N8SDR1/Lyra-SDR-cpp
SDROxide (dividebysandwich)	https://github.com/dividebysandwich/sdroxide
NereusSDR (boydsoftprez)	https://github.com/boydsoftprez/NereusSDR
Kainos (PNWHam-K7JUS)	https://github.com/PNWHam-K7JUS/Kainos
Bolt SDR (pe5jw)	https://github.com/pe5jw/bolt-sdr
Rigflow (dbourgoyne)	https://github.com/dbourgoyne/rigflow
OpenHPSDR Zeus (brianbruff) — archived	https://github.com/brianbruff/openhpsdr-zeus
AnanCore (ChasingCoffee)	https://github.com/ChasingCoffee/AnanCore
station-engine (Zeus-SDR)	https://github.com/Zeus-SDR/station-engine
ExpertSDR2 (closed source)	https://eesdr.com/
WDSP DSP library	https://github.com/TAPR/OpenHPSDR-wdsp

Conclusion: Is a Universal TCI Implementation Possible?

Short answer: Yes — with five concrete adaptations

A universal TCI client that works across all known server implementations is achievable. The core protocol is surprisingly consistent. The deviations are small in number but sharp in detail.

What we analysed

Ten implementations were analysed through direct source code (deskHPSDR `tci.c` at 6 200 lines, Zeus `TciSession.cs` at 2 100 lines) and the NereusSDR verification matrix which cites Thetis line-by-line. The central question was: **where do the real deviations live?**

The surprising outcome: most commands — `vfo`, `rx_filter_band`, `split_enable`, `rit_offset`, `sql_level`, `rx_nb_enable`, `rx_nr_enable`, `rx_anf_enable` — work identically across all implementations. The deviations are concentrated in five areas.

The five real deviations

1. `audio_start` — argument count

The ExpertSDR3 reference spec expects two arguments: `audio_start:trx,ch;`. Every other implementation expects one: `audio_start:rx;`. deskHPSDR actively rejects two arguments. This is the most direct breakage point for a universal client.

Fix: always send one argument. ExpertSDR3 is the only outlier and it is closed-source; the open-source community has already converged on one argument.

2. `trx` — broadcast or not

deskHPSDR and Zeus broadcast PTT state changes to all connected clients. Thetis and NereusSDR send the response **only** back to the requesting client. In a multi-client scenario (WSJT-X + logging software simultaneously) one client will be blind to the other's PTT state if it relies on broadcast.

Fix: actively query PTT state with `trx:0;` when in doubt, rather than relying on broadcast.

3. Modulation names — CW and FM

deskHPSDR sends `CW` for both CWL and CWU, and `FM` instead of `NFM`. All others send `CWL` / `CWU` and `NFM` (or accept both). Zeus sends `FM` but accepts `NFM` as an alias.

Fix: when receiving, accept `CW`, `CWL`, `CWU`, `FM` and `NFM` as all valid. When sending, use `CWU` — every implementation understands it.

4. `agc_mode` — command name and values

ExpertSDR3 uses the command `agc_auto_ex:rx,ch,bool;`. All open-source implementations (deskHPSDR, Thetis, NereusSDR, Zeus) use `agc_mode:rx,mode;` with named values. But the values differ too:

Implementation	AGC mode values
deskHPSDR	`off`, `fast`, `normal`
Zeus	`FIXED`, `SLOW`, `MEDIUM`, `FAST`, `CUSTOM`
Thetis / NereusSDR	`off`, `fast`, `normal`, `med` (med = normal)

Fix: use `agc_mode` with `fast` / `normal` / `off` — understood everywhere.

5. IQ streaming — ownership model

The most architectural deviation. deskHPSDR implements an *exclusive* ownership model: the first client to send `iq_start` owns the stream and sets the sample rate for everyone. If that client sends `iq_stop`, all IQ streams for all clients are stopped. Thetis and NereusSDR have not implemented this (it is a stub). Zeus uses a per-session subscription model without exclusivity.

Fix: with deskHPSDR, negotiate the sample rate before sending `iq_start`, and be careful with `iq_stop`.

Writing a universal client

A universal TCI client that works across all tested implementations follows these rules:

```
audio_start:0;              # 1 arg only — not 2
audio_samplerate:48000;     # harmless on deskHPSDR; required for Thetis/NereusSDR
iq_samplerate:192000;       # set BEFORE iq_start; deskHPSDR locks this rate exclusively
iq_start:0;

trx:0,true;                 # PTT on
trx:0,true,tci;             # PTT on + TX audio via WebSocket (deskHPSDR + Zeus)

modulation:0,CWU;           # send CWU — deskHPSDR accepts it, echoes CW back
                             # on receive: treat CW/CWL/CWU as equivalent

agc_mode:0,fast;            # works in all implementations
```

Detect the server via the init block and apply per-server adaptations:

```
protocol:ExpertSDR3,2.0; + device:Zeus;      → Zeus (full TCI; CWL/CWU correct; no PTT compat needed)
protocol:ExpertSDR3,1.7;                      → ExpertSDR3 (use 2-arg audio_start)
device:deskHPSDR;                             → deskHPSDR (CW alias; IQ exclusive ownership)
device:Thetis;                                → Thetis (audio/IQ stub; no PTT broadcast)
device:NereusSDR;                             → NereusSDR (SETUP: compat flags available)
```

With device detection, the five deviations can be handled in a small abstraction layer — roughly 200 lines of code.

Final verdict

A universal TCI implementation is achievable. The deviations are few and well-defined. The greatest risk is not syntax but behavioural: the deskHPSDR IQ ownership model and the missing PTT broadcast in Thetis/NereusSDR can cause silent failures that only surface when multiple clients connect simultaneously.

The open-source community has already informally converged away from the reference spec on `audio_start` — all active implementations use 1 argument. The ExpertSDR3 reference is trailing practice on that point.

Contributing

Corrections and additions are welcome. If you have verified a behaviour that is listed as `?` in the comparison table, or found a deviation not yet listed, please open a pull request or issue.

This document was assembled from public source code, protocol references, and repository documentation. It is not affiliated with Expert Electronics.